

Crewai Creating Raw Python Environment

Last amended: 19th May, 2026
My folder: D:\Documents\OneDrive\Documents\crewai
Reference: Crewai documentation here
Reference introductory videos on crewai here AND here.
See file: *python_packages installation.txt*

Stock Trading Agents: <https://github.com/siddhardhan2323/crewai-stock-trader-agents/tree/main>
<https://www.youtube.com/watch?v=2e7nhyAIsDk&t=1s>

Quick code for single python file:

s

Here is a quick 4-line code. But you can read on for explanation:

```
cd ~/
uv venv my_env                                # Create an env by name of my_env
source my_env/bin/activate                    # Activate it
# Note it is NOT 'pip install' but 'uv pip install'.
uv pip install crewai crewai[tools] langchain langchain-openai langchain-ollama
langchain-community langchain-classic langchain-experimental langchain-cli yfinance
uv pip install llama-index llama-index-llms-groq llama-index-core llama-index-read-
ers-file llama-index-embeddings-huggingface 'crewai[tools]' newsapi-python
```

Now you can create a python file in this environment and call crewai packages using 'from crewai import...'.

pip install vs uv pip install:

By default, 'uv pip install' requires an activated **virtual environment** and will refuse to install into the system Python unless you explicitly use the --system flag. 'pip install' will often install globally if no environment is active.

Quick code for standard crewai install:

Here is a quick 5+1-line code for standard crewai-CLI install with python packages added:

```
cd ~/
uv tool install crewai # Only if crewai was not earlier installed
                        # Execute anywhere no Python virtual env needed
crewai create crew my_project
cd my_project
crewai install
uv add crewai crewai[tools] langchain langchain-openai langchain-ollama
langchain-community langchain-classic langchain-experimental langchain-cli
yfinance
uv add llama-index llama-index-llms-groq llama-index-core llama-index-
readers-file llama-index-embeddings-huggingface 'crewai[tools]' newsapi-
python
# Execute to see what all is installed:
cat pyproject.toml
```

YAML syntax:

YAML (YAML Ain't Markup Language) syntax is built around human readability, using **indentation** and **key-value pairs** rather than the brackets or tags found in JSON or XML

Core Syntax Rules

- **Indentation:** Hierarchy is defined by spaces. You **cannot use tabs**; only literal spaces are allowed. Usually, 2 spaces per level is standard.

- **Key-Value Pairs:** Data is represented as `key: value`. A **space is required** after the colon.
- **Comments:** Any line starting with `#` is a comment and is ignored by parsers.

YAML automatically detects types like strings, integers, floats, booleans (`true/false`), and `null`. **Sequences (Lists):** Items in a list start with a hyphen and a space (`-`). In standard YAML syntax, there can be **any number of spaces** before the hyphen, including **zero space**. The hyphen is indented two spaces relative to the parent key. This is highly readable and standard practice. You must **always** include ONE space after the hyphen.

```
fruits:
  - Apple
  - Orange
```

- **Mappings (Dictionaries):** Nested objects are created by indenting new key-value pairs under a parent key.

```
person:
  name: John
  age: 30
```

- **Multi-line Strings:**
 - | (Literal): Preserves newlines exactly as written.
 - > (Folded): Replaces newlines with spaces, creating one long string.

::Q1:: How to create a raw python environment for crewai using uv?

To create a raw Python environment for **crewAI** using `uv`, you can initialize a new project, create a virtual environment, and then install the crewAI packages directly using `uv`'s high-speed package management commands.

Details

1. Install uv

If you haven't installed `uv` yet, use the following commands based on your operating system:

- **macOS/Linux:** `curl -Lsf https://astral.sh/uv/install.sh | sh`
- **Windows (PowerShell):** `powershell -c "irm https://astral.sh/uv/install.ps1 | iex"`

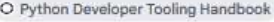
2. Initialize Your Project

Navigate to your desired folder and initialize a basic Python project structure:

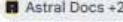
```
mkdir my-crew-project
cd my-crew-project
uv init
```

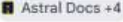
The `uv init` command sets up a new Python project by creating a standard folder structure and configuration files. It eliminates the hassle of manually creating virtual environments, `.gitignore` files, and metadata configurations. 



Depending on how you use it, `uv init` generates different types of project setups: 

1. Application Setup (`uv init`)

Running the command by itself (or with a directory name) creates a default application project perfect for web apps, scripts, and command-line tools. It automatically generates: 

- `pyproject.toml` : Manages project configuration and metadata.
- `README.md` : A placeholder for project documentation.
- `main.py` : A basic sample script to test your setup.
- `.python-version` : Specifies your target Python version.
- `.git` and `.gitignore` : Initializes version control. 

3. Create the Virtual Environment:

Create a managed virtual environment. For crewAI, it is recommended to use **Python 3.10 to 3.12**.

```
# Create environment with your system's default Python
uv venv

# OR specify a version (e.g., 3.12)
uv venv --python 3.12
```

To activate the environment:

- **macOS/Linux:** `source .venv/bin/activate`
- **Windows:** `.venv\Scripts\activate`

4. Install crewAI Packages:

Install the core library and any additional tools you might need using the `uv pip` interface:

```
# Install the main crewAI package in the virtual env
uv pip install crewai

# Install crewAI with built-in tools (recommended)
uv pip install 'crewai[tools]'
```

5. Verify Installation

Check that the environment is set up correctly by verifying the installed package version:

```
uv pip list | grep crewai
```

::Q2::After creating the virtual environment should I work in *my-crew-project* folder or *my-crew-project/.venv* folder

You should always work inside the **my-crew-project** folder. Here is why and how to handle both folders:

Where to Work

- **Keep your files here:** Write your code, configuration files, and script files directly inside **my-crew-project**.
- **Run commands here:** Execute your terminal commands (like `python main.py`) from this root folder.

What is the **.venv** Folder?

- **It is a system folder:** The `.venv` directory contains the Python interpreter, standard libraries, and downloaded dependencies (like `crewai`).
- **Do not modify it:** You should never manually add, edit, or delete files inside the `.venv` folder.
- **Hide it from Git:** Ensure `.venv` is added to your `.gitignore` file so you do not upload external dependencies to your repository.

::Q3:: Installation of crewai CLI does not install python packages?

That is completely correct. `uv tool install crewai` only installs the standalone CrewAI Command-Line Interface (CLI). It isolates the `crewai` binary globally so you can use commands like `crewai create crew`. It intentionally **does not** inject the Python libraries into your workspace. To actually write `from crewai import Agent` in a script, you must add the library package to your local environment.

How to Use the CrewAI Workflow Correctly?

You can resolve this by following the workflow that matches your development preference:

Option A: The Automated CrewAI Project Workflow (Recommended)

If you want to use CrewAI's structured template with pre-configured agents and tasks, use the CLI tool you just installed to generate the environment for you:

1. **Generate the project structure:**

```
crewai create crew my_project
```

2. **Navigate into the new folder:**

```
cd my_project
```

3. **Lock and install all Python libraries automatically:**

```
crewai install
```

(This command runs `uv pip install` behind the scenes inside a local virtual environment it creates for you).

4. Add any other python packages with `uv add`:

```
uv add crewai crewai[tools] langchain langchain-openai langchain-ollama
langchain-community langchain-classic langchain-experimental langchain-cli
yfinance llama-index llama-index-llms-groq llama-index-core llama-index-readers-
file llama-index-embeddings-huggingface newsapi-python
```

`uv add` uses `uv pip install` behind the scenes but also modifies

5. Point VS Code to this new environment:

Open the **Command Palette** (`Ctrl+Shift+P` / `Cmd+Shift+P`), **Python: Select Interpreter** select the `.venv` folder created inside `my_project`.

When you use the [CrewAI CLI](#) to create a project, it generates a standard structure centered around a `pyproject.toml` file. [LinkedIn · Fikayo Adepoju +3](#)

1. **pyproject.toml:** This file tracks your project's dependencies. When you run `uv add`, it automatically updates this file.
2. **Virtual Environment:** CrewAI automatically manages a virtual environment for your project. Using `uv` ensures that packages are installed specifically into this isolated environment rather than your global Python installation.
3. **Execution:** After installing new packages, you can run your project using `crewai run`, which executes the crew within the managed environment. [CrewAI Documentation +3](#)

Option B: The Quick Script Workflow (For single `.py` files)

If you just want to run a loose script without a formal project folder structure, you need to manually add the package to a local virtual environment:

6. Create a local virtual environment in your workspace:

```
uv venv          # Default env below: .venv
OR
uv venv my_env   # Env name: my_env
```

7. Activate the environment:

```
source .venv/bin/activate
OR
source my_env/bin/activate
```

8. Once, activated, install the Python package directly into this environment:

```
uv pip install crewai crewai[tools]
```

9. Match your VS Code Interpreter:

Ensure your VS Code Python Interpreter is pointing to the `.venv` (or `my_env`) file you just spun up in this folder.

10. Quick three lines code:

```
uv venv my_env
source my_env/bin/activate
pip install --upgrade pip
uv pip install crewai crewai[tools] langchain langchain-openai langchain-ollama langchain-community langchain-classic langchain-experimental langchain-cli
```

11. Remove my_env

```
rm -rf my_env
```

If you decide to go with **Option A**, let me know if you run into any dependency errors when running `crewai install`!

Q: How to add a SerperDevTool?

Here are the three steps:

1. Set the API Key in .env file

In your project's `.env` file, add your key: `SERPER_API_KEY=your_api_key_here`.

2. Declare the Tool in crew.py at the TOP of file

Instantiate the tool inside your main crew definition file—`crew.py`. Implement it at the **TOP** of file:

```
from crewai.tools import tool
from crewai_tools import SerperDevTool

# Inside your MyProjectCrew class:
@tool("Search tool")
def search_tool():
    """Useful to search the internet for web pages."""
    return SerperDevTool()
```

3. Mention the Tool in agents.yaml in list format

Mention it as:

```
researcher:
  role: >
    Senior Research Analyst
  goal: >
    Uncover cutting-edge developments in {topic}
  backstory: >
```

#####